12장
위치 정보 시각화하기: folium

이 장에서는 위치 정보를 시각화하는 방법을 배워 보겠습니다. 위치 정보를 시각화할 때도 적절한 예를 찾으려고 했지만 파이썬에서는 `folium` 등과 같이 편리한 라이브러리를 제공해 주는데 위치 정보를 쉽고 시원시원하게 표현할 수 있게 해주니 정말 시각화하기 좋겠죠?

12.1

지도 만들기

12.1.1 folium 설치하고 지도 만들기

먼저 `folium`을 설치해 봅시다. `folium`은 파이썬에 내장된 라이브러리가 아니어서 따로 설치해야 합니다. 주피터 노트북에서는 `!pip install folium`으로 설치합니다. 코랩에서는 따로 설치할 필요 없습니다.

```
!pip install folium
```

실행결과

```
Collecting folium
  Downloading folium-0.12.1-py2.py3-none-any.whl (94 kB)
…
Successfully installed branca-0.4.2 folium-0.12.1
```

설치 내용이 나오다가 가장 마지막 줄에 'Successfully installed...'라는 메시지가 보이면 설치가 끝납니다(버전은 책과 다를 수 있습니다). 이제 `folium` 라이브러리를 자유롭게 사용할 수 있습니다.

먼저 위치 정보를 표시할 지도를 만들어 봅시다. 지도는 `folium` 라이브러리의 `Map([위도, 경도])` 명령어로 만들 수 있습니다. 나타내고 싶은 위치의 위도, 경도를 명령어에 넣어 주면 해당 위치를 중심으로 지도가 만들어집니다. 위치를 넣지 않으면 세계지도를 기본으로 만듭니다. `folium` 라이브러리에 속한 명령어를 사용하므로 실제 사용할 때는 `folium.Map([위도, 경도])`라고 작성합니다.

형식

```
folium.Map([위도, 경도])
```

지도에 여의도 한강공원을 표시해 봅시다. 여의도 한강공원의 위도와 경도를 알아야겠죠. 위도, 경도는 구글 지도에서 알아낼 수 있습니다. 그림을 보면서 차례대로 따라 해 보세요. 구글에서 여의도 한강공원을 검색한 후 검색창 바로 아래에 있는 지도 탭을 클릭합니다.



▲ 그림 12-1 구글 지도에서 장소 검색하기

여의도 한강공원이 구글 지도 위에 빨간 마커로 표시됩니다. 마커(maker)란 다음 그림처럼 지도 위에 특정 지점을 표시하는 데 사용하는 핀 모양 이미지입니다. 마커에 마우스를 놓고 우클릭하면 팝업 메뉴가 뜨고 가장 위에 숫자 2개가 나타납니다. 이 숫자가 여의도 한강공원의 위도와 경도입니다.



▲ 그림 12-2 여의도 한강공원의 위도와 경도 구하기

위도는 37.52860, 경도는 126.93431로 나오네요. 마커의 어떤 부분을 클릭했는지에 따라 위도와 경도는 소수점 넷째 자리부터 조금씩 다를 수 있습니다. 하지만 진행하는 데는 문제없으니 각자 화면에 보이는 숫자를 입력해 진행합니다.

이제 알아낸 위도와 경도를 명령어에 넣습니다. 위도와 경도는 리스트로 묶어야 오류가 발생하지 않습니다. 그리고 문자열, 숫자형, 리스트, 딕셔너리 등의 데이터를 변수에 저장한 것처럼 `folium.Map([위도, 경도])`로 만든 지도 역시 변수에 저장할 수 있습니다. 여의도 한강공원을 표시하는 지도이므로 여의도의 y을 따서 변수명을 `map_y`로 하겠습니다. 그리고 지도를 화면에 출력하기 위해 지도가 저장된 `map_y` 변수를 한 번 더 작성하고 실행합니다.

```
import folium

map_y = folium.Map([37.52860, 126.93431])
map_y
```

실행결과



지도가 나타났지만, 여의도 한강공원이 안 보이죠? 지도 왼쪽 상단에 있는 + 버튼을 계속 클릭하면 비로소 여의도 한강공원이 보입니다. 사실 입력한 좌표로 지도가 정확히 표현되긴 했으나 지도 비율이 맞지 않아서 보이지 않았던 겁니다. 우리가 원하는 정도로 보려면 지도를 확대해야 하죠.

지도 비율 조절하기
지도의 확대 및 축소를 조절하는 옵션은 `zoom_start`입니다. 옵션을 추가해 봅시다.

```
import folium

map_y = folium.Map([37.52860, 126.93431], zoom_start=15)
map_y
```

실 행 결 과



이제는 여의도 한강공원이 지도에서 바로 보이네요. `zoom_start` 옵션의 숫자가 커질수록 지도가 확대됩니다. + 버튼을 누르는 효과죠. 반대로 숫자가 작을수록 지도가 축소(- 버튼 효과)됩니다.

이번에는 앞의 코드를 조금 수정해 보겠습니다. 지금은 예제이니 `folium.Map([37.52860, 126.93431])`처럼 위도와 경도를 명령어에 바로 넣을 수 있지만 보통은 위도와 경도가 별도의 변수에 저장되는 경우가 많습니다. 영어로 위도는 latitude, 경도는 longitude이므로 위도는 변수 `lat`에, 경도는 변수 `long`에 저장하고 실행해 보겠습니다.

```
lat, long = 37.52860, 126.93431
map_y = folium.Map([lat, long], zoom_start=15)
map_y
```

실행하면 앞의 코드와 동일한 결과가 나옵니다. 이처럼 변수를 활용하는 방식에 익숙해지면 프로그래밍하기가 훨씬 수월해집니다.

마커 표시하기

앞의 지도에 마커를 추가해 보겠습니다. 마커를 만드는 명령어는 `Marker([위도, 경도])`입니다. `Map()`과 마찬가지로 `folium`에 속한 명령어이므로 실제 사용할 때는 `folium.Marker([위도, 경도])`라고 작성합니다. 그리고 여기에 마커를 표시할 지도 이름을 `.add_to(지도)`로 추가합니다. 그럼 앞에서 만든 지도에 여의도 한강공원을 마커로 표시해 봅시다.

```
lat, long = 37.52860, 126.93431
map_y = folium.Map([lat, long], zoom_start=15)
folium.Marker([lat, long]).add_to(map_y)
map_y
```

실행결과



여의도 한강공원에 마커가 표시됩니다. 이때 마커의 위도와 경도는 지도를 만들 때 사용한 위도, 경도와 같지 않아도 됩니다. `folium.Map([위도, 경도])`의 위도와 경도는 지도를 만들 때 사용하는 기준 좌표일 뿐입니다. 다른 좌표에 마커를 표시해도 됩니다.

12.1.2 여러 위치 표시하기

이번에는 지도에 여러 위치를 표시해 보겠습니다. 여의도 한강공원과 여의도공원을 표시해 보죠. 구글 지도에서 여의도공원을 검색한 후, 위도와 경도를 알아냅니다(마우스를 우클릭한 위치에 따라 소수점 넷째자리부터 책과 다를 수 있습니다).



▲ 그림 12-3 여의도공원의 위도와 경도 구하기

앞의 지도에 여의도 한강공원은 이미 표시되어 있으니 여의도공원만 추가하면 됩니다. 앞의 코드에 여의도공원 마커를 추가해 봅시다. 위도와 경도가 2개이므로 변수명을 `lat1`, `long1`, `lat2`, `long2`로 변경한 후 진행합니다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker([lat1, long1]).add_to(map_y)
folium.Marker([lat2, long2]).add_to(map_y)
map_y
```

실행결과



지도에 마커 2개가 잘 보입니다.

마커에 텍스트 표시하기

마커가 여러 개일 때 마커가 각각 어디를 나타내는지 정보를 보여 주면 더 좋겠죠? 이럴 때 사용할 수 있는 옵션이 있는데, `tooltip`과 `popup`입니다. `tooltip`은 마커에 마우스를 올리면 텍스트가 나타나고 마우스를 옮기면 텍스트가 바로 사라집니다. `popup`은 마커를 클릭할 때 말풍선 모양으로 텍스트가 나타나고 마커를 다시 클릭하면 텍스트가 사라집니다.

`tooltip`부터 실습해 봅시다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker([lat1, long1], tooltip='여의도 한강공원').add_to(map_y)
folium.Marker([lat2, long2], tooltip='여의도공원').add_to(map_y)
map_y
```

실행결과



마커에 마우스를 올렸다 내렸다 하면서 텍스트가 나타났다 사라지는지 확인해 보세요.

NOTE 툴팁에서 한글이 깨져 보인다면

다음 설명대로 실행하면 한글이 잘 보입니다.

1. 다음 코드를 실행해 패키지를 설치합니다.

```
!pip install git+https://github.com/python-visualization/branca.git@master
```

2. 메뉴에서 [런타임 → 런타임 다시 시작]을 클릭합니다.
3. 한글이 포함된 코드를 재실행합니다.

이번에는 `tooltip`을 `popup`으로 바꾸어 실행해 봅시다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker([lat1, long1], popup='여의도 한강공원').add_to(map_y)
folium.Marker([lat2, long2], popup='여의도공원').add_to(map_y)
map_y
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

마커를 클릭하면 마우스 움직임과 상관없이 설정한 텍스트가 고정적으로 표시됩니다. 다시 마커를 클릭하면 텍스트는 사라집니다. 이처럼 지도에 여러 위치를 표시할 때는 `tooltip`과 `popup`을 활용해 마커에 정보를 넣을 수 있습니다.

위치를 하나 더 추가해 봅시다. 여의도의 또 다른 공원인 샛강생태공원을 구글 지도에서 찾으면 위도 37.51865, 경도 126.92041로 나옵니다. 샛강생태공원의 위도와 경도를 `lat3`, `long3`에 저장하고 마커를 추가합니다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원
lat3, long3 = 37.51865, 126.92041 # 샛강생태공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker([lat1, long1], tooltip='여의도 한강공원').add_to(map_y)
folium.Marker([lat2, long2], tooltip='여의도공원').add_to(map_y)
folium.Marker([lat3, long3], tooltip='샛강생태공원').add_to(map_y)
map_y
```

실행결과



마커 3개와 툴팁이 잘 보입니다.

마커의 아이콘과 색 변경하기

이번에는 마커에 아이콘과 색을 넣어 구분해 보겠습니다. 마커의 아이콘과 색 변경은 `icon=folium.Icon()`으로 합니다. 아이콘 모양은 설정하지 않으면 기본으로 `info-sign`(i 모양)이 됩니다. 아이콘 색상은 `color` 옵션으로 설정해 `icon=folium.Icon(color='색상 이름')`처럼 작성합니다. 이렇게 색상을 설정한 아이콘을 `folium.Marker()`에 추가합니다.

앞의 지도에서 각 마커의 색상을 `red`, `blue`, `purple`로 설정하고 코드를 실행해 봅시다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원
lat3, long3 = 37.51865, 126.92041 # 샛강생태공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker(
    [lat1, long1], tooltip='여의도 한강공원', icon=folium.Icon(color='red')
).add_to(map_y)
folium.Marker(
    [lat2, long2], tooltip='여의도공원', icon=folium.Icon(color='blue')
).add_to(map_y)
folium.Marker(
    [lat3, long3], tooltip='샛강생태공원', icon=folium.Icon(color='purple')
).add_to(map_y)
map_y
```

```
실행결과
```



Copyright © Gilbut, Inc. All rights reserved.

아이콘 모양이 `info-sign`으로 바뀌고 각 마커가 3가지 색으로 표시됩니다. 위치를 더 명확하게 구분할 수 있습니다.

TIP 마커에 설정할 수 있는 색상은 다음과 같습니다.

```
'red', 'blue', 'green', 'purple', 'orange', 'darkred', 'lightred', 'beige', 'darkblue',
'darkgreen', 'cadetblue', 'darkpurple', 'white', 'pink', 'lightblue', 'lightgreen', 'gray',
'black', 'lightgray'
```

마커의 아이콘을 다른 모양으로 변경해 보겠습니다. 아이콘 모양은 아이콘 색상과 마찬가지로 `folium.Icon()`에 `icon` 옵션으로 설정합니다. 바꿀 수 있는 모양은 `heart`, `star`, `flag`, `bookmark`, `home` 등이 있습니다. 아이콘 모양을 `heart`(하트), `home`(집), `flag`(깃발)로 변경해 봅시다.

```
lat1, long1 = 37.52860, 126.93431 # 여의도 한강공원
lat2, long2 = 37.52400, 126.91889 # 여의도공원
lat3, long3 = 37.51865, 126.92041 # 샛강생태공원

map_y = folium.Map([lat1, long1], zoom_start=15)
folium.Marker(
    [lat1, long1], tooltip='여의도 한강공원',
    icon=folium.Icon(color='red', icon='heart')
).add_to(map_y)
folium.Marker(
    [lat2, long2], tooltip='여의도공원',
    icon=folium.Icon(color='blue', icon='home')
).add_to(map_y)
folium.Marker(
    [lat3, long3], tooltip='샛강생태공원',
    icon=folium.Icon(color='purple', icon='flag')
).add_to(map_y)
map_y
```

실 행 결 과

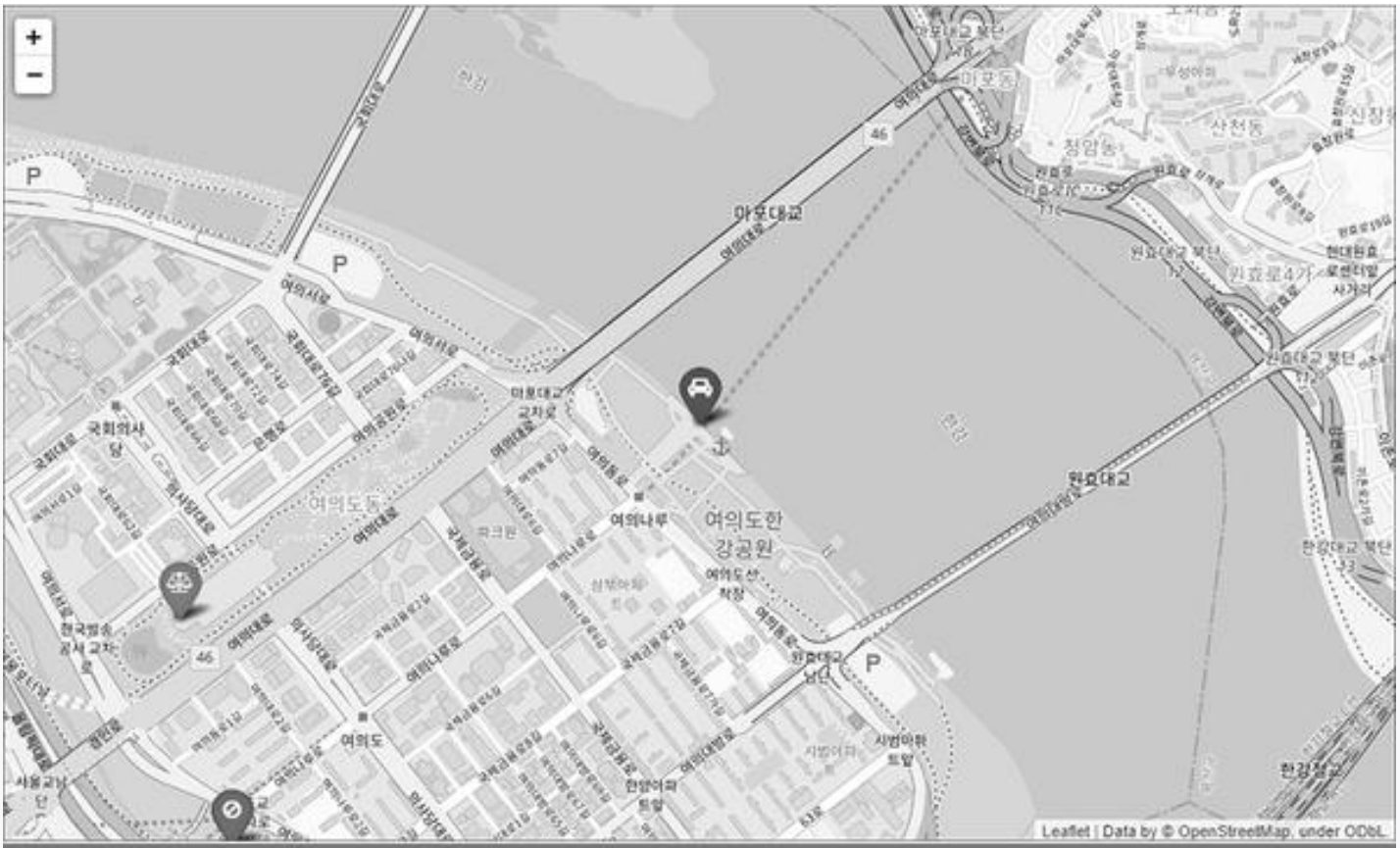


아이콘이 i에서 하트, 집, 깃발 모양으로 변경됐습니다. 옵션을 추가하면 외부에서 더 많은 아이콘을 가져와 사용할 수 있습니다. 대표적으로 fontawesome 사이트가 있습니다. https://fontawesome.com/v4.7.0/icons에 가면 `folium`에서 적용할 수 있는 다양한 아이콘이 있습니다. https://fontawesome.com으로 가면 최신 버전으로 연결되니 반드시 URL에 /v4.7.0/icons를 붙여 주세요. 최신 버전은 설정되지 않는 아이콘과 유료 아이콘이 많아 추천하지 않습니다.

이 중에서 `automobile`(자동차), `balance-scale`(저울), `ban`(금지)을 아이콘으로 설정해 봅시다. 아이콘 모양을 설정하는 방법은 동일하나 fontawesome 사이트의 아이콘을 사용한다는 의미로 `prefix='fa'`를 표시해야 합니다.

```
folium.Marker(
    [lat1, long1], tooltip='여의도 한강공원',
    icon=folium.Icon(color='red', icon='automobile', prefix='fa')
).add_to(map_y)
folium.Marker(
    [lat2, long2], tooltip='여의도공원',
    icon=folium.Icon(color='blue', icon='balance-scale', prefix='fa')
).add_to(map_y)
folium.Marker(
    [lat3 , long3], tooltip='샛강생태공원',
    icon=folium.Icon(color='purple', icon='ban', prefix='fa')
).add_to(map_y)
map_y
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

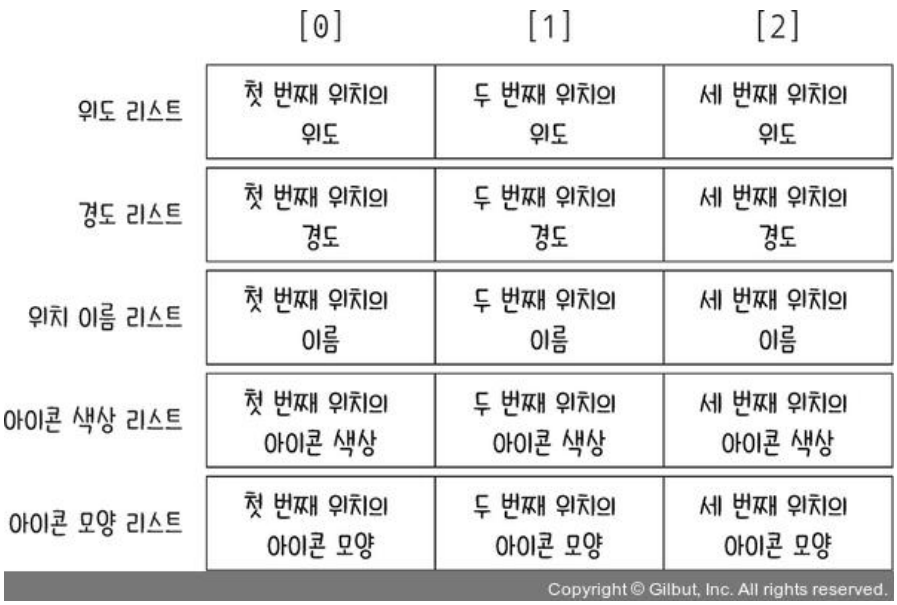
반복문으로 마커 표시하기

이번에는 반복문을 사용해 여러 개의 마커를 표시해 보겠습니다. 코드를 보면 `folium.Marker().add_to()` 부분이 마커 개수만큼 반복되는 것을 알 수 있습니다. 이 부분을 반복문으로 만들면 다음과 같습니다.

```
for i in range(마커개수):
    folium.Marker(
        [위도, 경도], tooltip='위치 이름',
        icon=folium.Icon(color='아이콘 색상', icon='아이콘 모양', prefix='fa')
        ).add_to(지도이름)
```

코드에서 반복할 때마다 변하는 것은 위도, 경도, 위치 이름, 아이콘 색상, 아이콘 모양입니다. 따라서 다섯 가지 요소를 리스트로 각각 만들고, 각 리스트의 인덱스를 활용합니다. 예를 들어, 첫 번째 반복에서는 5개의 리스트의 첫 번째 요소([0])가 자동으로 위도, 경도, 위치 이름, 아이콘 색상, 아이콘 모양에 들어가고, 두 번째 반복에서는 각 리스트의 두 번째 요소([1])가 자동으로 들어가면 되죠.

다음처럼 총 5개의 리스트를 만들 수 있습니다. 위도 리스트의 첫 번째 요소와 경도 리스트의 첫 번째 요소를 결합하면 첫 번째 마커의 좌표가 됩니다.



▲ 그림 12-4 위도, 경도, 위치 이름, 아이콘 색상, 아이콘 모양으로 리스트 만들기

이와 마찬가지로 위치 이름 리스트, 아이콘 색상 리스트, 아이콘 모양 리스트의 첫 번째 요소가 첫 번째 마커의 이름과 아이콘 속성에 해당합니다. 두 번째 마커를 나타내려면 5개 리스트의 두 번째 요소들이 모이면 되고요.




```
for i in range(마커개수):
    folium.Marker(
        [위도리스트[i], 경도리스트[i]], tooltip=위치이름리스트[i],
        icon=folium.Icon(color=아이콘색상리스트[i], icon=아이콘모양리스트[i], prefix='fa')
    ).add_to(지도이름)
```

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 12-5 리스트의 코드 대입

그럼 코드로 작성해 봅시다. 지도를 만들려면 기준 위치가 필요하니 지도를 그리는 명령어에 첫 번째 위치의 위도, 경도를 넣고 시작합니다.

```
import folium

lat = [37.52860, 37.52400, 37.51865]                        # 위도 리스트
long = [126.93431, 126.91889, 126.92041]                    # 경도 리스트
names = ['여의도 한강공원', '여의도공원', '샛강생태공원']   # 위치 이름 리스트
icons = ['automobile', 'balance-scale', 'ban']              # 아이콘 모양 리스트
colors = ['red', 'blue', 'purple']                          # 아이콘 색상 리스트

map_y = folium.Map([lat[0], long[0]], zoom_start=15)
for i in range(len(lat)):
    folium.Marker(
        [lat[i], long[i]], tooltip=names[i],
        icon=folium.Icon(color=colors[i], icon=icons[i], prefix='fa')
        ).add_to(map_y)
map_y
```

실행결과



위치 개수가 적을 때는 반복문을 사용하지 않아도 되지만, 표시할 위치가 늘어나면 반복문을 사용하는 것이 효율적입니다. 반복문과 리스트를 활용하는 방법은 다른 곳에서도 자주 사용하니 잘 알아 두기 바랍니다.

12.2

지도에 프로젝트 결과 표시하기

그럼 앞서 다룬 핫플레이스 인구 분석 프로젝트와 `folium`을 접목해 봅시다.

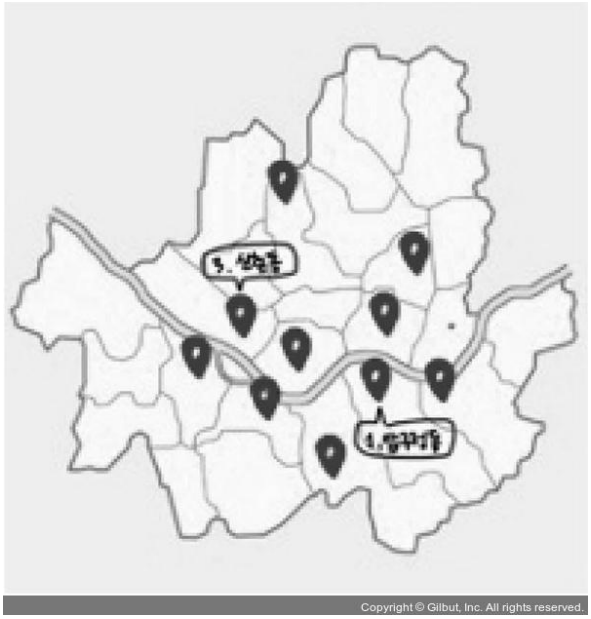
12.2.1 목표 수립하기

핫플레이스 인구 분석 프로젝트에서는 4개의 하위 목표를 세워 프로그램을 구현했습니다. 여기에 하위 목표 하나를 추가하겠습니다.

앞에서 수집한 인구 데이터를 통해 생활인구가 많은 행정동을 순서대로 구할 수 있습니다. 그래서 이번에는 '인구가 많은 상위 10개 행정동을 뽑아 지도에 표시하기'를 하위 목표로 추가해 프로그램을 작성해 봅시다. 해당 지역을 마커로 표시하고 `tooltip`으로 행정동명도 나타냅니다. 목표를 간단히 스케치해 보겠습니다.

하위 목표 5

인구가 많은 상위 10개 행정동을 뽑아 지도에 표시하기



프로그램을 어떻게 작성할지 한번 생각해 봅시다. 일단 행정동별 인구수를 모두 구해야 합니다. 카드 이용내역에서 매출액 상위 10개 가맹점을 뽑을 때처럼 행정동명을 키로, 인구수를 값으로 하는 딕셔너리를 만들면 됩니다. 그리고 딕셔너리를 인구수 기준으로 내림차순 정렬하고 앞에서 10개 요소만 선택하면 인구수가 가장 많은 행정동 10개가 추려집니다. 딕셔너리 정렬은 `sorted()`와 `operator` 라이브러리의 `itemgetter()`와 `reverse=True` 옵션을 사용하면 됩니다.

이렇게 인구수 상위 10개 지역을 추렸다면 `folium`으로 해당 지역을 지도 위에 표시합니다. 지도에 위치를 표시하려면 위도와 경도를 알아야 하죠. 그런데 우리가 받은 파일에는 인구수만 나와 있을 뿐이고 행정동의 위도와 경도 정보는 없습니다. 따라서 행정동의 위도와 경도를 알려 주는 새로운 데이터를 구해야 합니다.

인구 데이터를 내려받은 서울 열린데이터 광장 사이트로 가 봅시다.



▲ 그림 12-6 행정동 위치 정보 검색하기

'행정동 위치 정보'를 검색하면 다양한 검색결과가 나오지만, 우리가 원하는 위치 정보만 있는 데이터는 없어 보입니다. 하지만 상심하지 마세요. 이런 데이터에도 위도, 경도가 포함되어 있으므로 이를 활용하면 됩니다. '서울시 행정동별 전력 사용량 2008년 위치정보(좌표계: WGS1984)'를 클릭해 봅시다.

스크롤을 내려 보면 하단에 데이터 미리보기가 있어서 내려받기 전에 데이터 항목을 확인할 수 있습니다. 미리보기를 보니 행정동코드와 경도, 위도가 포함되어 있네요. 그럼 이 데이터를 CSV 파일로 내려받아 봅시다. 그리고 LOCAL_PEOPLE_DONG_201912.csv와 dong_code.csv 파일이 있는 폴더에 내려받고, 파일명을 영문으로 변경합니다. 이번에는 dong_location.csv로 하겠습니다.

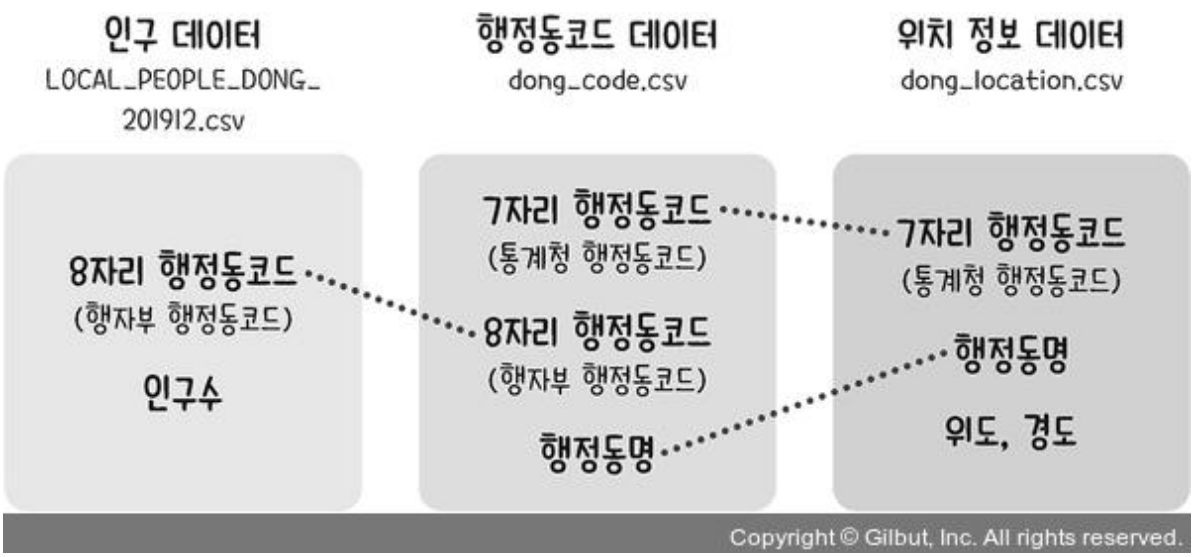


고유번호	기준연도	행정동코드	행정동명	2007년사용량	2008년사용량	경도	위도
3	2008	1103058	효창동	20.814328	21.459975	126.960713	37.5436889
6	2008	1101071	숭인2동	72.809621	74.027617	127.0195273	37.5749267
8	2008	1102052	소공동	423.513379	424.742285	126.9746802	37.5644921
9	2008	1105062	구의3동	133.85004	131.664698	127.0961417	37.5357391
103	2008	1111059	하계2동	36.930365	37.577989	127.0646016	37.6351662

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 12-7 공공 데이터의 CSV 파일 내려받기

그런데 dong_location.csv 파일을 살펴보니 행정동코드가 7자리입니다. 핫플레이스 프로젝트에서 dong_code.csv 파일을 다룰 때 행정동코드가 7자리와 8자리, 2가지가 있었죠. 인구 데이터는 8자리 행정동코드가 기준이었습니다. 이처럼 행정동코드가 다를 때는 어떻게 해야 할까요? 이때는 7자리 코드와 8자리 코드를 모두 가지고 있는 파일을 활용하면 됩니다. 파일별로 가지고 있는 정보를 정리하면 다음과 같습니다.

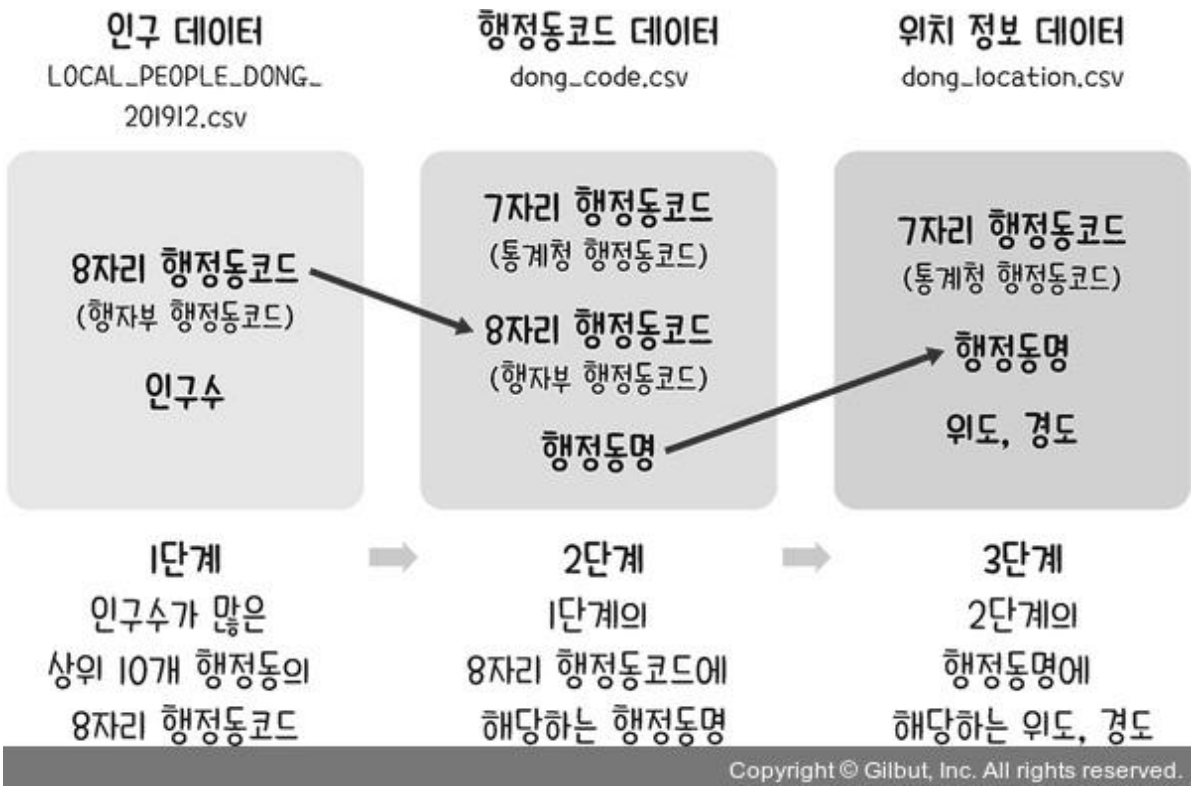


▲ 그림 12-8 데이터의 연결 관계 1

인구 데이터 파일과 행정동코드 데이터 파일은 익숙하죠? 인구 데이터는 8자리 행정동코드를 사용하므로 입력된 행정동명에 해당하는 8자리 코드를 행정동코드 데이터에서 알아낸 후 반영했습니다.

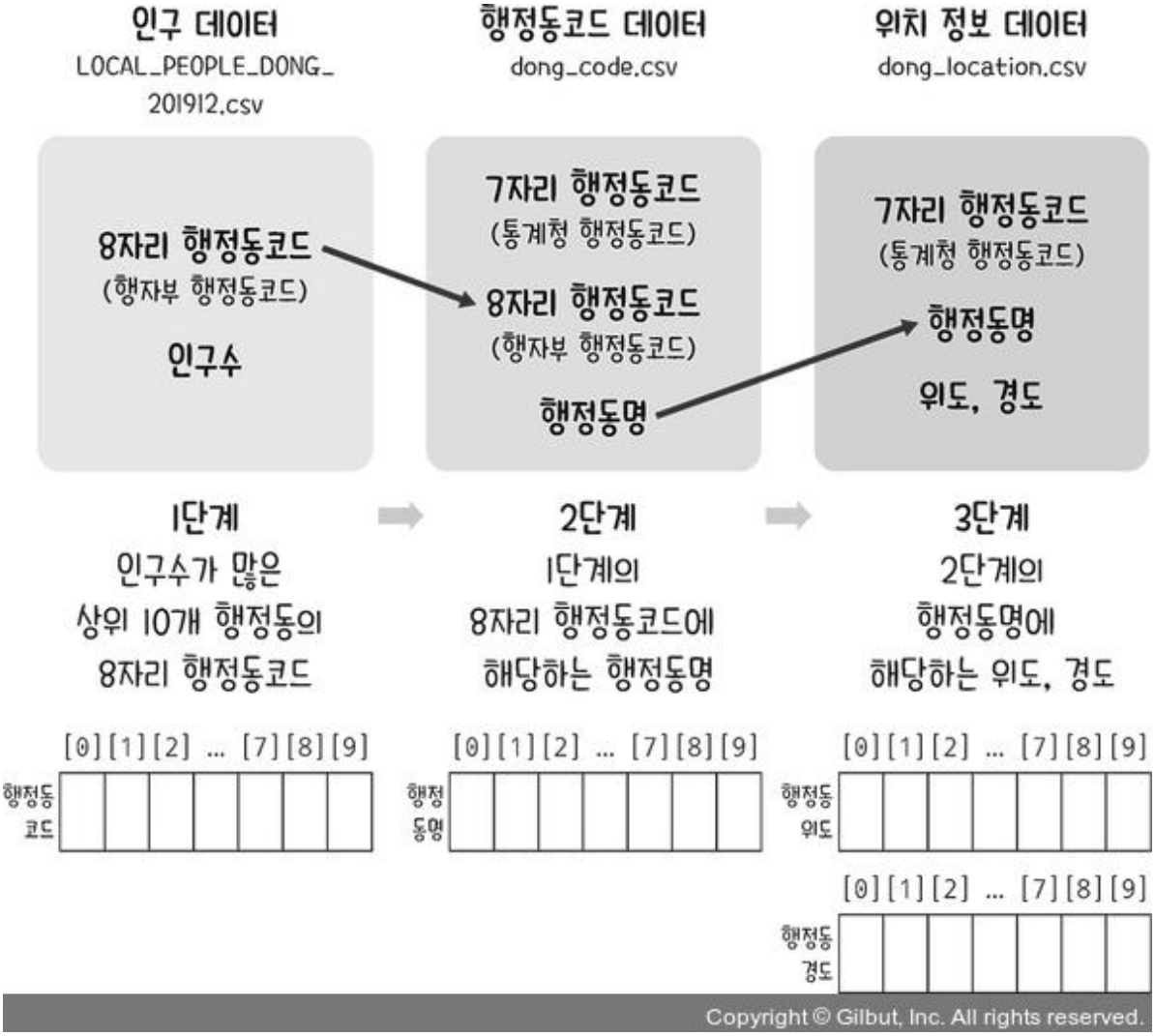
추가된 dong_location.csv 파일에는 7자리 행정동코드와 행정동명, 위치 정보가 포함되어 있습니다. 같은 항목끼리 연결해 보면 파일 간 연결 관계가 드러납니다. 인구 데이터와 위치 데이터는 공통 항목이 없지만, 행정동코드 데이터는 인구 데이터, 위치 데이터와 각각 접점이 있으니 이를 활용합니다.

우선 1단계에서는 인구 데이터를 분석해 인구가 많은 상위 10개 지역의 8자리 행정동코드를 알아냅니다. 그리고 2단계에서는 행정동코드 데이터에서 8자리 행정동코드에 해당하는 행정동명을 구합니다. 마지막 3단계에서는 위치 데이터에서 행정동명의 위도, 경도를 알아내면 됩니다. 그림으로 정리해 볼까요? 화살표와 추가된 텍스트를 잘 보세요.



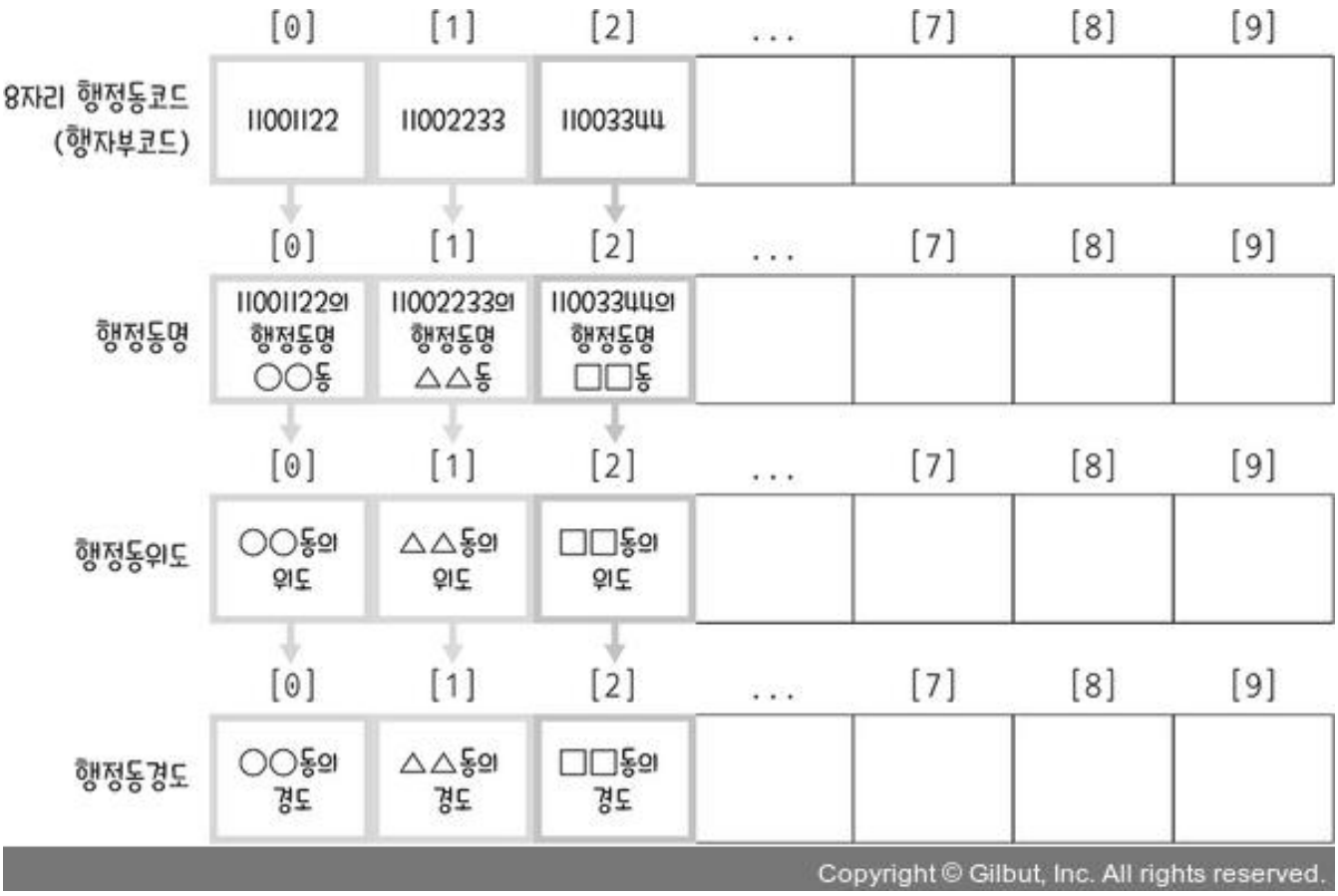
▲ 그림 12-9 데이터의 연결 관계 2

연결 관계를 보면 인구 데이터와 위치 데이터를 이어주는 행정동코드 데이터가 매우 중요한 역할을 합니다. 행정동을 10개 골라야 하므로 1단계에서 얻어야 하는 행정동코드도 10개입니다. 이를 리스트로 묶으면 효율적입니다. 2단계에서도 행정동명이 10개 나오면 이를 묶어 하나의 리스트로 만듭니다. 마지막 단계에서도 마찬가지로 위도 10개, 경도 10개를 각각 묶어 리스트로 만들면 됩니다. 단계별로 만들어지는 리스트까지 정리하면 다음과 같습니다.



▲ 그림 12-10 데이터의 연결 관계 3

`행정동코드[0]`에 해당하는 행정동명은 `행정동명[0]`에 있습니다. 위도와 경도는 `행정동위도[0]`, `행정동경도[0]`에 있고요. 이와 마찬가지로 `행정동코드[1]`에 저장된 지역의 정보도 다른 리스트의 인덱스 [1]에 있습니다. 글로 표현하면 조금 복잡해 보이지만 앞에서 지도에 여러 위치를 표시할 때 리스트를 여러 개 만든 적이 있으니 참고하면 이해하기 쉬울 겁니다.



▲ 그림 12-11 리스트들의 인덱스 호응관계

그림을 보면 리스트의 인덱스를 맞춘다는 의미를 이해할 겁니다. `행정동코드[0]`에 저장된 행정동코드가 11001122이라면, 11001122의 행정동명은 `행정동명[0]`에, 위도와 경도는 각각 `행정동위도[0]`, `행정동경도[0]`에 저장됩니다. 이처럼 행정동명과 위치 정보를 구하는 과정은 다른 인덱스에도 동일하게 적용돼야 하므로 이 과정을 리스트 요소 개수대로 10번 반복합니다.

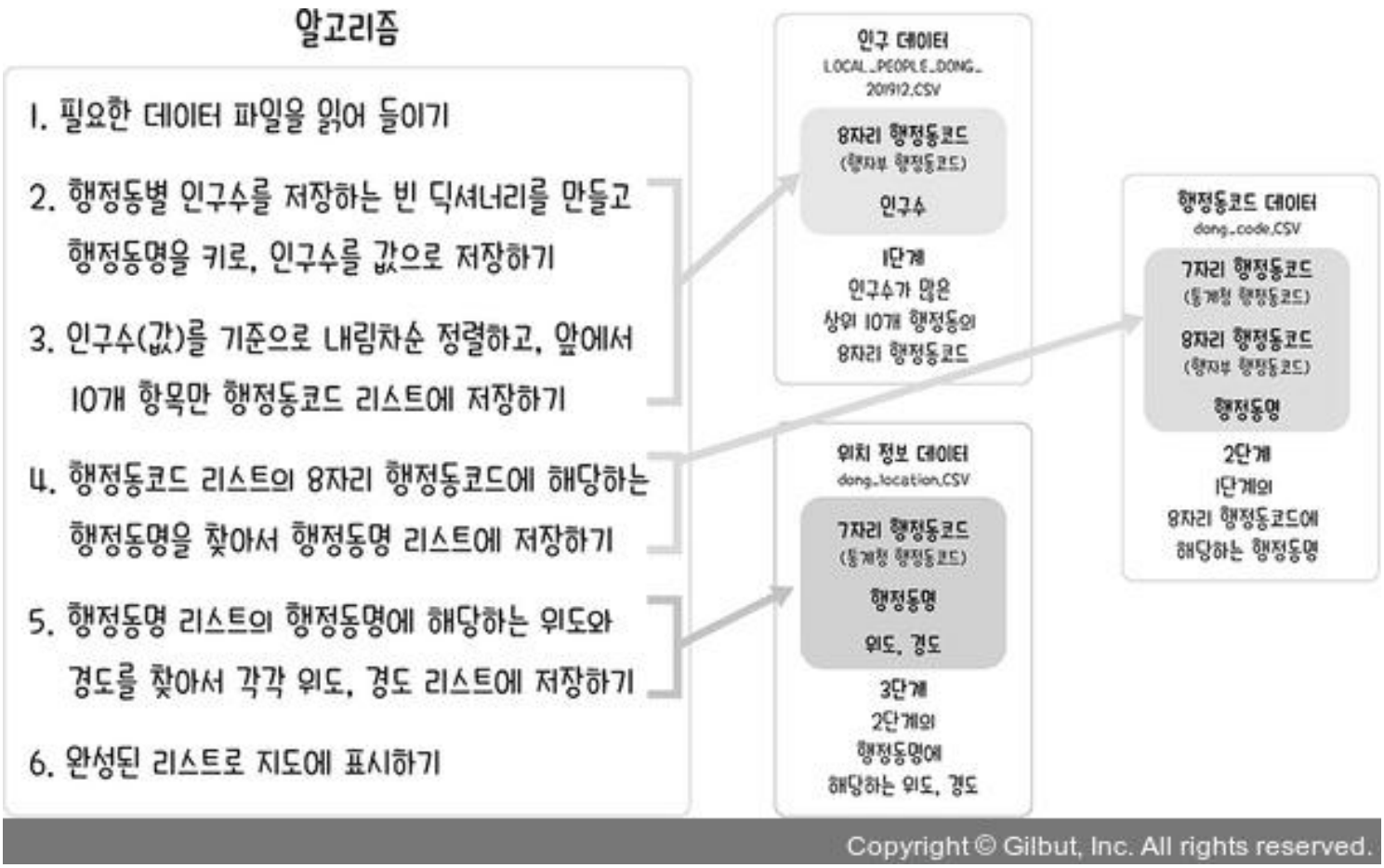
12.2.2 프로그램으로 구현하기

지금까지 설명한 내용을 바탕으로 프로그램을 구현해 봅시다.

알고리즘 작성하기

하위 목표 5

인구가 많은 상위 10개 행정동을 뽑아 지도에 표시하기



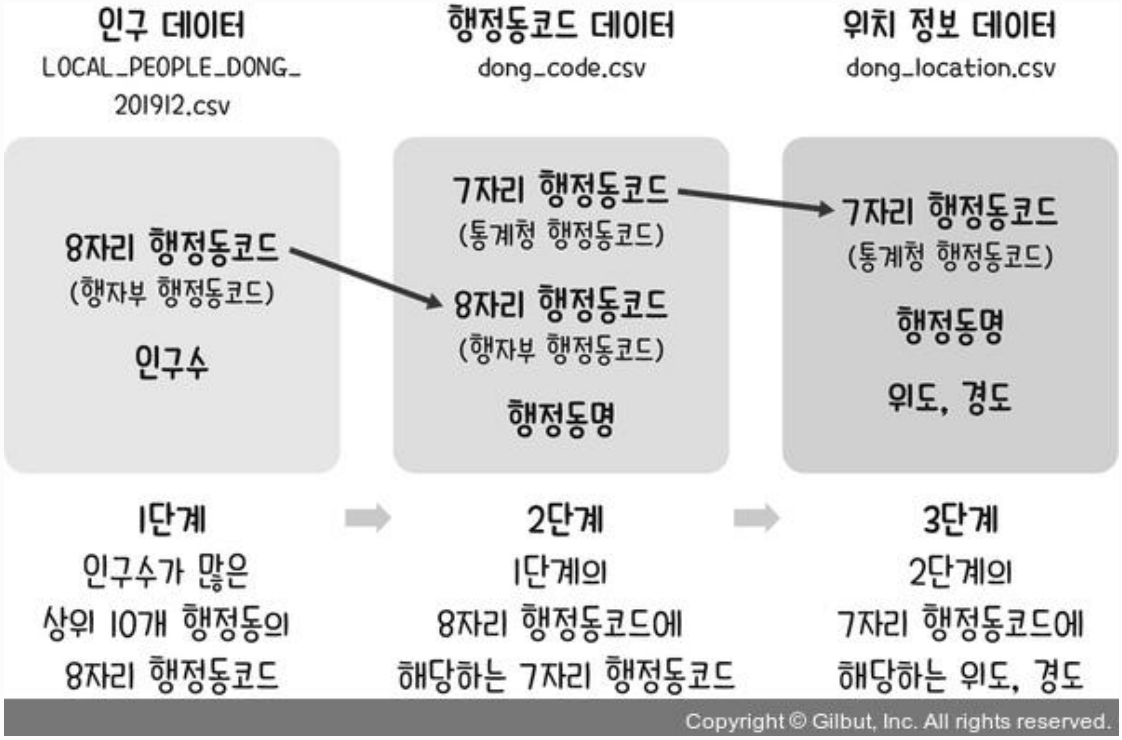
▲ 그림 12-12 알고리즘

우선 알고리즘부터 작성해 봅시다.

앞에서 설명한 내용을 단계에 맞춰 정리하면 자연스럽게 알고리즘이 완성됩니다. 처음에 파일을 읽어 들여 빈 딕셔너리를 만드는 과정을, 마지막에 완성된 리스트를 지도에 표시하는 과정을 추가했습니다.

NOTE 7자리 행정동코드로 연결하기

행정동코드보다는 행정동명이 더 알기 쉽기 때문에 책에서는 행정동명을 매개로 위치 정보를 찾지만, 7자리 행정동코드를 매개로 위치 정보를 찾아도 됩니다. 행정동명과 마찬가지로 7자리 행정동코드 역시 행정동코드 데이터와 위치 정보 데이터, 둘 다에 있습니다.



▲ 그림 12-13 데이터의 연결 관계 4

프로그램을 작성하다 보면 행정동명을 매개로 하는 것과 7자리 행정동코드를 사용하는 것이 서로 보완적이라는 것을 알게 될 겁니다. 이는 뒤에서 차근차근 살펴보겠습니다.

코드 작성하기

작성한 알고리즘을 코드로 구현해 봅시다. 먼저 과정 1과 2를 코드로 옮겨 봅시다. 알고리즘을 참고해서 먼저 작성해 본 후 이 책의 코드를 보세요.

사용하는 데이터 파일이 3개이므로 CSV 파일을 여는 변수와 데이터를 저장하는 변수들이 겹치지 않게 정합니다. 인구 데이터는 `data`, 행정동코드 데이터는 `code_data`, 위치 데이터는 `location_data`에 리스트로 저장합니다. 11장의 파일 읽기 코드에 위치 데이터를 읽어 들이는 코드를 추가하면 됩니다.

```
# 1. 필요한 데이터 파일 모두 읽어 들이기
import csv
# 코랩의 파일 업로드
# from google.colab import files
# uploaded = files.upload()

# 인구 데이터 읽어와서 리스트로 저장하기
f1 = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8')
data = csv.reader(f1)
next(data)
data = list(data)

# 행정동코드 데이터 읽어와서 리스트로 저장하기
f2 = open('dong_code.csv', encoding='utf8') # 또는 'cp949'
code_data = csv.reader(f2)
next(code_data)
next(code_data)
code_data = list(code_data)

# 위치 정보 데이터 읽어와서 리스트로 저장하기
f3 = open('dong_location.csv')
location_data = csv.reader(f3)
next(location_data)
location_data = list(location_data)

# 인구 데이터의 데이터 타입 변환하기
for row in data:
    for i in range(1, 32):
        if i <= 2:
            row[i] = int(row[i])
        else:
            row[i] = float(row[i])

# 행정동코드 데이터의 데이터 타입 변환하기
for row in code_data:
    row[0], row[1] = int(row[0]), int(row[1]) # 7자리 행정동코드, 8자리 행정동코드

# 위치 정보 데이터의 데이터 타입 변환하기
for row in location_data: # row[2]: 행정동코드, row[-2]: 경도, row[-1]: 위도
    row[2], row[-2], row[-1] = int(row[2]), float(row[-2]), float(row[-1])
```

코드는 길지만, 모두 배운 내용이기 때문에 이해하기 어려운 부분은 없을 겁니다. 11장에서는 행정동코드 파일(dong_code.csv)에서 8자리 행정동코드만 변환했지만, 여기에서는 7자리 행정동코드도 데이터 타입을 변환합니다. 그 이유는 뒤에서 확인합니다. 위치 데이터에서도 필요한 항목인 행정동코드, 경도, 위도를 숫자로 변경했습니다. 다음 과정으로 넘어가 봅시다.

> 제제쌤의 조언
> dong_location.csv 파일을 읽어 들일 때도 인코딩 옵션에 유의해 주세요. Windows라도 PC 세부 사양에 따라 인코딩 옵션을 지정하지 않거나 utf8, euc-kr, cp949 등을 지정해야 하는 경우도 있습니다. 오류가 발생하더라도 당황하지 말고 다른 옵션을 넣어서 실행해 보세요.

과정 2에서는 행정동별 인구수를 저장하는 `dong_population` 딕셔너리를 만듭니다. 그리고 인구 데이터를 담은 `data`를 돌며 행정동코드(열 인덱스 [2])를 키로, 총생활인구수(열 인덱스 [3])를 값으로 해서 한 쌍씩 `dong_population`에 저장합니다(`data`의 항목은 11장을 참고하거나 직접 파일을 열어서 확인해 보세요).

```
# 2. 행정동별 인구수를 저장하는 빈 딕셔너리를 만들고
#    행정동별 총생활인구수를 딕셔너리에 저장하기
dong_population = {}

for row in data:
    dong_code, p = row[2], row[3]
    if dong_code not in dong_population.keys():
        dong_population[dong_code] = p
    else:
        dong_population[dong_code] += p

# print(dong_population)
```

실행결과

```
{11215710: 32699578.380499963, 11680545: 36303123.67050001, 11680610: 39427473.14730003... }
```

비어 있던 `dong_population` 딕셔너리에 데이터가 저장됩니다.

코드를 한 줄씩 살펴봅시다. 먼저 반복문으로 `data`를 돌며 행정동코드(`row[2]`)와 총생활인구수(`row[3]`)를 변수 `dong_code`와 `p`에 각각 저장합니다. 이때 해당 행의 `dong_code`를 키로 하는 항목이 없다면 {키:값}이 `{dong_code:p}`인 항목을 `dong_population` 딕셔너리에 추가합니다. 만약 `dong_code`를 키로 하는 항목이 있다면 해당 키의 값에 현재 행의 인구수 `p`를 더합니다. 결과를 보면 `dong_code`가 11215710인 행정동은 2019년 12월 한 달간 총 생활인구수가 약 32,699,578명입니다(결과가 길게 출력되므로 한 번만 확인하고 그 이후에는 주석 처리하거나 삭제해도 됩니다).

이제 과정 3에서 총인구수를 기준으로 딕셔너리를 내림차순 정렬할 차례입니다.

```
# 3. 인구수를 기준으로 내림차순 정렬하고, 앞에서 10개 항목만 별도로 저장하기
import operator

top10 = sorted(dong_population.items(), key=operator.itemgetter(1),
reverse=True)[:10]
# print(top10)

top10_code8 = []
for t in top10:
    top10_code8.append(t[0])
print(top10_code8)
```

```
실행결과
[11680640, 11560540, 11440660, 11110615, 11650530, 11410585, 11380690, 11545510, 11680650, 11
680610]
```

딕셔너리의 키-값 쌍으로 정렬해야 하므로 `dong_population.items()`를 사용합니다. 그리고 `operator`의 `itemgetter()`를 사용해 값을 기준으로 내림차순 정렬합니다. 이 중 앞에서 10개 항목을 선택해야 하므로 `[:10]`을 추가해 슬라이싱합니다. 이 상태로 변수 `top10`에 저장된 값을 출력하면(`print(top10)`) 딕셔너리의 키-값이 괄호로 묶여 리스트 형태로 출력됩니다(주석을 지우고 출력해 보세요). 필요한 것은 행정동코드이므로 첫 번째 항목(키에 해당하는 행정동코드)만 뽑아서 다시 `top10_code8` 리스트에 차례대로 저장합니다. `top10`에 저장된 순서대로 `top10_code8`에 저장되므로 `top10_code8`에도 인구가 많은 행정동코드부터 차례대로 추가됩니다.

TIP 이해되지 않으면 7.1.4 다양한 명령어 사용하기의 딕셔너리 정렬하기: sorted()를 참고하세요.

그런데 행정동코드를 `top10_code8`에 저장하는 과정에서 반복문을 사용하네요. 11장에서 배운 리스트 내포를 사용하면 코드를 한 줄로 바꿀 수 있습니다. 리스트 내포를 사용해 코드를 다음과 같이 수정합니다.

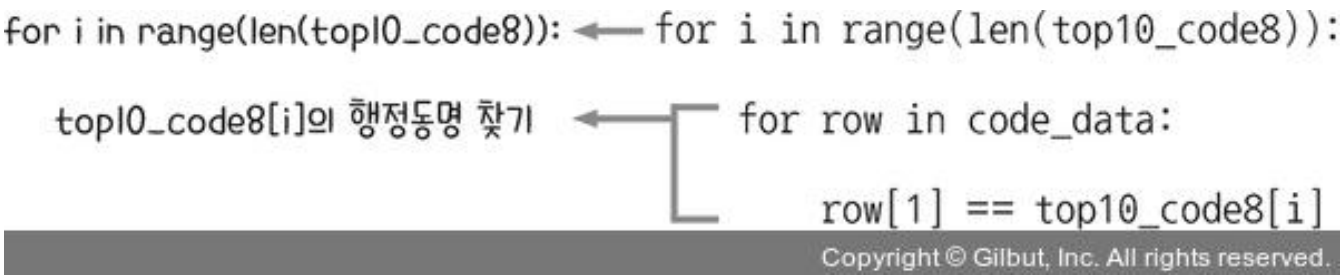
```
top10 = sorted(dong_population.items(), key=operator.itemgetter(1), reverse=True)[:10]
top10_code8 = [t[0] for t in top10]
```

다음 과정으로 넘어갑시다. 과정 4에서는 `top10_code8` 리스트에 저장한 행정동코드에 해당하는 행정동명을 행정동코드 데이터에서 찾아 행정동명 리스트에 저장합니다. 그리고 과정 5에서는 행정동명에 해당하는 위도와 경도를 찾아 위도와 경로 리스트에 각각 저장합니다.

먼저 필요한 리스트를 만듭니다. 행정동명을 저장하는 리스트는 `top10_name`, 위도와 경도를 저장하는 리스트는 각각 `top10_lat`, `top10_long`으로 이름을 정합니다. 10개 지역이므로 각각 초깃값 0을 넣어 10칸짜리 리스트로 만듭니다.

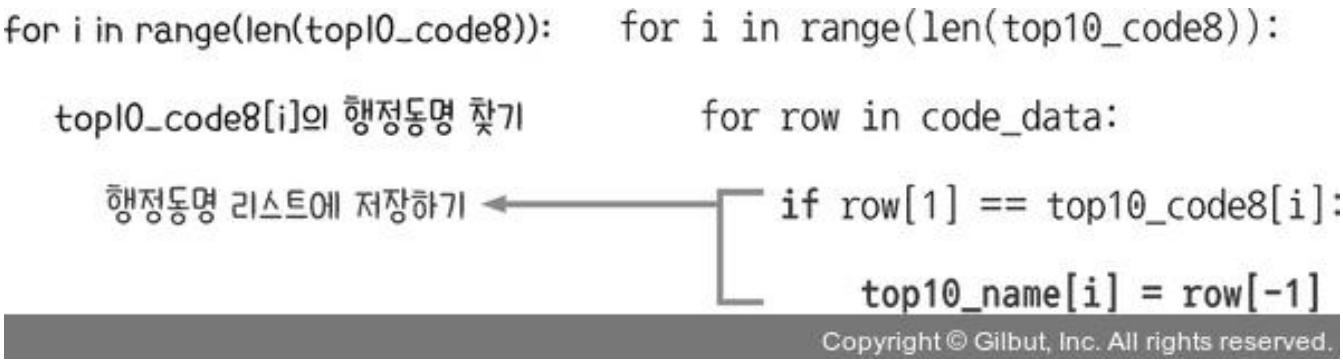
```
top10_name = [0 for i in range(10)]
top10_lat = [0 for i in range(10)]
top10_long = [0 for i in range(10)]
```

리스트를 만들었으니 데이터를 넣어야죠. `top10_name` 리스트에는 `top10_code8` 리스트의 행정동코드에 해당하는 행정동명을 찾아 저장합니다. 이는 `top10_code8[0]`부터 `top10_code8[9]`까지의 행정동명을 모두 찾아야 한다는 뜻입니다. 반복되는 부분이 보이죠? `top10_code8[i]`의 행정동명 찾기가 반복되므로 다음처럼 반복문을 작성할 수 있습니다.



`range(len(top10_code8))`로 반복 범위를 설정합니다. 반복문에서 `range(len(리스트명))`을 사용하면 자동으로 인덱스 [0]부터 인덱스 [리스트길이-1](리스트의 마지막)까지 반복 범위가 된다고 배웠습니다.

그 다음으로 '행정동명 찾기'인데 8자리 행정동코드가 주어졌으므로 dong_code.csv 데이터를 담은 `code_data`를 활용합니다. `code_data`에서 행정동코드는 열 인덱스 [1]에 있고, 행정동명은 열 인덱스 [-1]에 담겨 있습니다. 따라서 행정동코드(`top10_code8[i]`)와 `code_data`의 열 인덱스 [1]의 값으로 행정동명(열 인덱스 [-1])을 구하면 됩니다(`code_data`의 각 요소에 들어 있는 항목은 11장을 참고하세요).



이렇게 찾은 행정동명을 행정동명 리스트인 `top10_name`에 저장하면 됩니다. 인덱스가 같도록 `top10_name[i]`에 저장합니다.

```
# 4. 8자리 행정동코드에 해당하는 행정동명 찾기
for i in range(len(top10_code8)):
    for row in code_data:
        if row[1] == top10_code8[i]:
            top10_name[i] = row[-1] # 행정동명 리스트에 저장하기
```

반복문 안에 반복문, 또 그 안에 조건문이 있는 형태네요. 이처럼 제어구조가 중첩될 때는 앞에서와 같이 단계를 나눠 생각하는 것이 좋습니다. 인덱스를 의미하는 `i`가 변할 때마다 행정동명을 찾는 반복문이 실행됩니다.

다음 과정으로 행정동명에 해당하는 위도, 경도를 구해야 합니다. 위도, 경도를 구하는 코드 역시 반복문 안에 들어가야 합니다.

```
for i in range(len(top10_code8)):
  top10_code8[i]의 행정동명을 찾아
    행정동명 리스트에 저장하기
  행정동명에 해당하는 위도, 경도 찾기
```

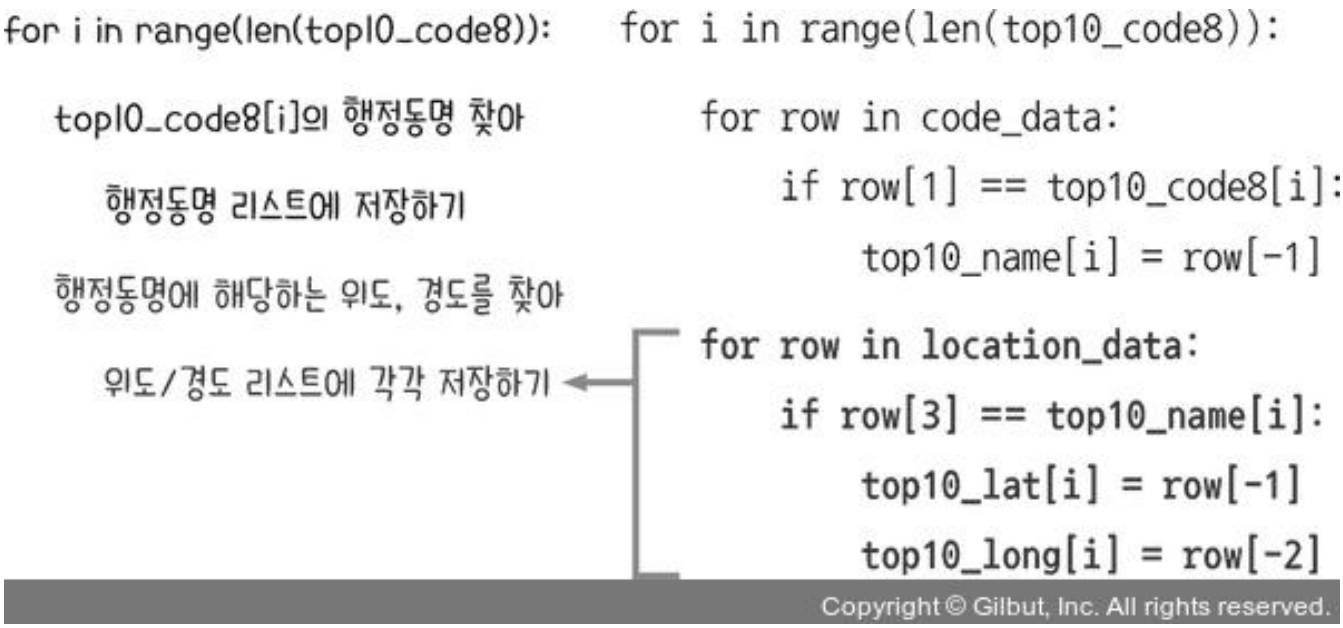
행정동명을 활용해 위도와 경도를 찾아야 하므로 행정동명을 찾는 코드 아래에 작성합니다. 위도와 경도 찾기는 dong_location.csv 파일을 담고 있는 `location_data` 리스트를 활용합니다. dong_location.csv 파일의 항목은 다음과 같습니다.

고유번호	기준연도	행정동코드	행정동명	2007년사용량	2008년사용량	경도	위도
3	2008	1103058	효창동	20.814328	21.459975	126.960713	37.5436889
6	2008	1101071	숭인2동	72.809621	74.027617	127.0195273	37.5749267
8	2008	1102052	소공동	423.513379	424.742285	126.9746802	37.5644921
9	2008	1105062	구의3동	133.85004	131.664698	127.0961417	37.5357391
103	2008	1111059	하계2동	36.930365	37.577989	127.0646016	37.6351662

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 12-14 dong_location.csv 파일의 데이터 항목

행정동명이 네 번째 항목에 있습니다. 즉, `location_data` 리스트의 열 인덱스 [3]이 행정동명입니다. 따라서 `location_data`의 열 인덱스 [3]과 앞에서 구한 `top10_name[i]`이 같은 행의 위도와 경도가 찾는 값입니다(`location_data`는 경도, 위도 순서로 되어 있으니 유의하세요). 위도(열 인덱스 [-1])는 위도 리스트(`top10_lat`)의 인덱스 `[i]`에 넣고, 경도(열 인덱스 [-2])는 경도 리스트(`top10_long`)의 인덱스 `[i]`에 각각 넣습니다. 이를 코드로 작성하면 다음과 같습니다.



과정 5의 추가 코드는 다음과 같습니다. 마지막에 위도와 경도 리스트의 값을 출력해 직접 확인해 봅시다.

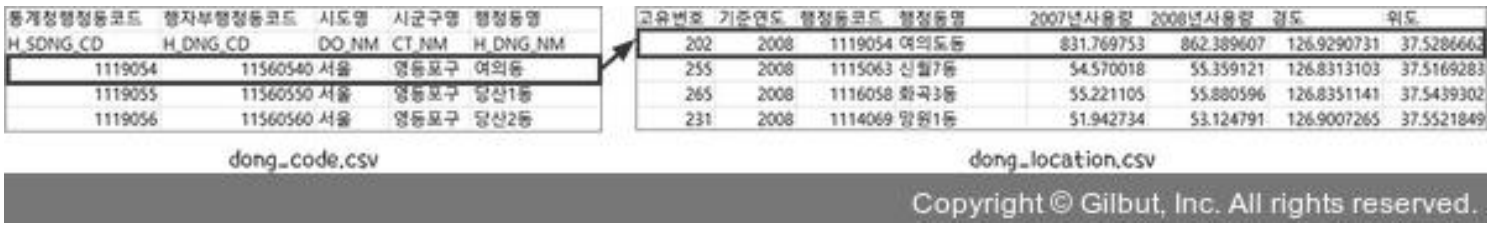
```
# 4. 8자리 행정동코드에 해당하는 행정동명 찾기
for i in range(len(top10_code8)):
    for row in code_data:
        if row[1] == top10_code8[i]:
            top10_name[i] = row[-1] # 행정동명 리스트에 저장하기
    # 5. 행정동명에 해당하는 위도와 경도를 찾아 리스트에 저장하기
    for row in location_data:
        if row[3] == top10_name[i]:
            top10_lat[i], top10_long[i] = row[-1], row[-2]
print(top10_lat)
print(top10_long)
```

실행결과

```
[37.4999211, 0, 37.5550337, 37.5777701, 37.4856591, 37.5659638, 37.64193, 37.4756389, 37.4984
733, 37.4955294]
[127.0367584, 0, 126.9228533, 126.9892737, 127.0097616, 126.9384315, 126.9387277, 126.885072
1, 127.0449272, 127.0662436]
```

위도와 경도 리스트가 출력됩니다. 그런데 두 리스트의 인덱스 [1]의 값이 0으로 나옵니다. 다른 9개 값은 잘 저장됐는데 인덱스 [1]만 다를까요?

답은 데이터에 있습니다. 과정 3 코드를 실행한 결과에서 두 번째 행정동코드는 11560540입니다. `code_data`(dong_code.csv)에서 8자리 행정동코드가 11560540인 곳은 '여의동'입니다. 따라서 `top10_name[i]`에 '여의동'이 저장됩니다. 그런데 `location_data`(dong_location.csv)에는 여의동이 아닌 '여의도동'으로 저장되어 있습니다. dong_location.csv가 오래전에 작성된 파일이라서 그간의 행정동명 변화가 반영되어 않아서 그렇습니다.



통계청행정동코드	행자부행정동코드	시도명	시군구명	행정동명
H_SDNG_CD	H_DNG_CD	DO_NM	CT_NM	H_DNG_NM
1119054	11560540	서울	영등포구	여의동
1119055	11560550	서울	영등포구	당산1동
1119056	11560560	서울	영등포구	당산2동

dong_code.csv

고유번호	기준연도	행정동코드	행정동명	2007년사용량	2008년사용량	경도	위도
202	2008	1119054	여의도동	831.769753	862.389607	126.9290731	37.5286662
255	2008	1115063	신월7동	54.570018	55.359121	126.8313103	37.5169283
265	2008	1116058	화곡3동	55.221105	55.880596	126.8351141	37.5439302
231	2008	1114069	망원1동	51.942734	53.124791	126.9007265	37.5521849

dong_location.csv

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 12-15 데이터가 일치하지 않는 경우(행정동명이 바뀐 경우)

이렇게 위치 정보가 누락된 경우에는 7자리 행정동코드를 매개로 찾습니다. 데이터 파일에서 7자리 행정동코드가 1119054로 같은 것이 보이죠. 그래서 위도 리스트와 경도 리스트의 값 중에서 하나라도 0인 요소는 7자리 행정동코드로 다시 위치 정보를 찾게 합니다.

```
# 추가. 행정동명으로 위치 정보를 찾지 못하면 7자리 행정동코드로 찾기
for i in range(len(top10_lat)):
    # 위도나 경도가 0인 요소가 있다면
    if top10_lat[i] == 0 or top10_long[i] == 0:
        # code_data에서 7자리 행정동코드를 구해 dong7에 저장하기
        for row in code_data:
            if top10_code8[i] == row[1]:
                dong7 = row[0]
        # location_data에서 7자리 행정동코드가 dong7인 행을 찾아 위도와 경도를 구하기
        for row in location_data:
            if dong7 == row[2]:
                top10_lat[i], top10_long[i] = row[-1], row[-2]
print(top10_lat)
print(top10_long)
```

실행결과

```
[37.4999211, 37.5286662, 37.5550337, 37.5777701, 37.4856591, 37.5659638, 37.64193, 37.475638
9, 37.4984733, 37.4955294]
[127.0367584, 126.9290731, 126.9228533, 126.9892737, 127.0097616, 126.9384315, 126.9387277, 1
26.8850721, 127.0449272, 127.0662436]
```

`top10_lat[1]`과 `top10_long[1]`에 정상적인 값이 저장됐습니다. 데이터 파일이 서로 잘 맞으면 작성하지 않아도 될 코드입니다. 사실 데이터 분석을 진행하다 보면 데이터 자체에 문제가 있는 경우가 상당히 많습니다.

TIP 행정동명이 일치하지 않으면 위도와 경도 모두 값이 제대로 저장되지 않습니다. 그래서 `if top10_lat[i] == 0 or top10_long[i] == 0` 대신 `if top10_lat[i] == 0` 또는 `if top10_long[i] == 0`으로 작성해도 이 프로그램에서는 같은 결과가 나옵니다.

NOTE 7자리 행정동코드를 매개로 사용한다면

'처음부터 7자리 행정동코드를 매개로 위치 정보를 찾았다면 추가 코드가 필요 없지 않았을까?' 하는 의문이 들 수 있습니다. 여기서 행정동명을 매개로 한 이유는 이후 지도를 그릴 때 마커의 `tooltip` 옵션에서 행정동명을 설정하므로 행정동명을 미리 저장해 두는 것이 편하기 때문입니다. 그리고 7자리 행정동코드를 매개로 코드를 작성해도 행정동 1개의 위치 정보는 찾지 못합니다. 강남구 대치2동은 7자리 행정동코드 자체가 변경됐습니다.



통계청행정동코드	행자부행정동코드	시도명	시군구명	행정동명
H_SDNG_CD	H_DNG_CD	DO_NM	CT_NM	H_DNG_NM
1123079	11680610	서울	강남구	대치2동
1123080	11680670	서울	강남구	개포2동
1124051	11710510	서울	송파구	풍납1동

dong_code.csv

고유번호	기준연도	행정동코드	행정동명	2007년사용량	2008년사용량	경도	위도
343	2008	1123061	대치2동	60.524873	61.273872	127.0662436	37.4955294
344	2008	1123070	개포3동	31.428367	31.704273	127.0718191	37.4885825
347	2008	1123066	도곡1동	119.655308	123.822486	127.0407387	37.4894403
349	2008	1124051	풍납1동	53.800824	49.416853	127.1137027	37.5382922

dong_location.csv

Copyright © Gilbut, Inc. All rights reserved.

▲ 그림 12-16 데이터가 일치하지 않는 경우(행정동코드가 바뀐 경우)

지도 그리기

이제 `folium`으로 지도를 그려 보죠. 10개 장소를 마커로 표시해야 하니 앞서 여러 위치를 지도에 한 번에 표시할 때 배운 리스트와 반복문을 활용합니다. 지도 이름은 `map_top10`으로 하고 지도를 만드는 좌표는 위도, 경도 리스트의 첫 번째 요소로 하겠습니다. 마커의 아이콘과 색상은 각자 원하는 대로 설정하세요.

```
# 6. 완성된 리스트로 지도 그리기
import folium

map_top10 = folium.Map([top10_lat[0], top10_long[0]], zoom_start=12)
for i in range(10):
    folium.Marker(
        [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
        icon=folium.Icon(color='red', icon='star')
    ).add_to(map_top10)
map_top10
```

실행결과



행정동 10곳에 마커가 뜹니다. 마커에 마우스를 올려서 행정동명도 직접 확인해 보세요.

지도에 순위가 표시되면 좋겠죠? 어렵지 않습니다. 행정동명을 저장한 `top10_name` 리스트가 이미 인구가 많은 순서대로 저장되어 있기 때문이죠. '1. △△동'처럼 인구가 많은 순서대로 1부터 10까지 순위를 붙여 봅시다. 인덱스는 0부터 시작하므로 순위는 인덱스에 1을 더한 값입니다. 따라서 `top10_name`의 인덱스에 1을 더한 값을 문자로 바꾸어 `top10_name[i]`의 이름과 합치면 됩니다.

순위를 표시하는 코드까지 추가해 전체 코드를 정리하면 다음과 같습니다.

```
# 1. 필요한 데이터 파일 모두 읽어 들이기
import csv

# 인구 데이터 읽어와서 리스트로 저장하기
f1 = open('LOCAL_PEOPLE_DONG_201912.csv', encoding='utf8')
data = csv.reader(f1)
next(data)
data = list(data)

# 행정동코드 데이터 읽어와서 리스트로 저장하기
f2 = open('dong_code.csv', encoding='utf8') # 또는 'cp949'
code_data = csv.reader(f2)
next(code_data)
next(code_data)
code_data = list(code_data)
```

```
# 위치 정보 데이터 읽어와서 리스트로 저장하기
f3 = open('dong_location.csv')
location_data = csv.reader(f3)
next(location_data)
location_data = list(location_data)

# 인구 데이터의 데이터 타입 변환하기
for row in data:
    for i in range(1, 32):
        if i <= 2:
            row[i] = int(row[i])
        else:
            row[i] = float(row[i])

# 행정동코드 데이터의 데이터 타입 변환하기
for row in code_data:
    row[0], row[1] = int(row[0]), int(row[1])

# 위치 정보 데이터의 데이터 타입 변환하기
for row in location_data:
    row[2], row[-2], row[-1] = int(row[2]), float(row[-2]), float(row[-1])

# 2. 행정동별 인구수를 저장할 수 있게 빈 딕셔너리를 만들고
#    행정동별 총생활인구를 딕셔너리에 저장하기
dong_population = {}
for row in data:
    dong_code, p = row[2], row[3]
    if dong_code not in dong_population.keys():
        dong_population[dong_code] = p
    else:
        dong_population[dong_code] += p
```

```
# 3. 인구수를 기준으로 내림차순 정렬하고, 앞에서 10개 항목만 별도로 저장하기
import operator

top10 = sorted(dong_population.items(), key=operator.itemgetter(1), reverse=True)[:10]
top10_code8 = [t[0] for t in top10]

top10_name = [0 for i in range(10)]
top10_lat  = [0 for i in range(10)]
top10_long = [0 for i in range(10)]

# 4. 8자리 행정동코드에 해당하는 행정동명 찾기
for i in range(len(top10_code8)):
    for row in code_data:
        if row[1] == top10_code8[i]:
            top10_name[i] = row[-1]   # 행정동명 리스트에 저장하기
    # 5. 행정동명에 해당하는 위도와 경도를 찾아 리스트에 저장하기
    for row in location_data:
        if row[3] == top10_name[i]:
            top10_lat[i], top10_long[i] = row[-1], row[-2]

# 추가. 행정동명으로 위치 정보를 찾지 못하면 7자리 행정동코드로 찾기
for i in range(len(top10_lat)):
    # 위도나 경도가 0인 요소가 있다면
    if top10_lat[i] == 0 or top10_long[i] == 0:
    # code_data에서 7자리 행정동코드를 구해 dong7에 저장하기
    for row in code_data:
        if top10_code8[i] == row[1]:
            dong7 = row[0]
    # location_data에서 7자리 행정동코드가 dong7인 행을 찾아 위도와 경도 구하기
    for row in location_data:
        if dong7 == row[2]:
            top10_lat[i], top10_long[i] = row[-1], row[-2]
```

```
# 6. 완성된 리스트로 지도 그리기
import folium

# 행정동명 앞에 순위 표시하기
for i in range(len(top10_name)):
    top10_name[i] = str(i + 1) + '. ' + top10_name[i]

map_top10 = folium.Map([top10_lat[0], top10_long[0]], zoom_start=12)
for i in range(10):
    folium.Marker(
        [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
        icon=folium.Icon(color='red', icon='star')
    ).add_to(map_top10)
map_top10
```

실행결과

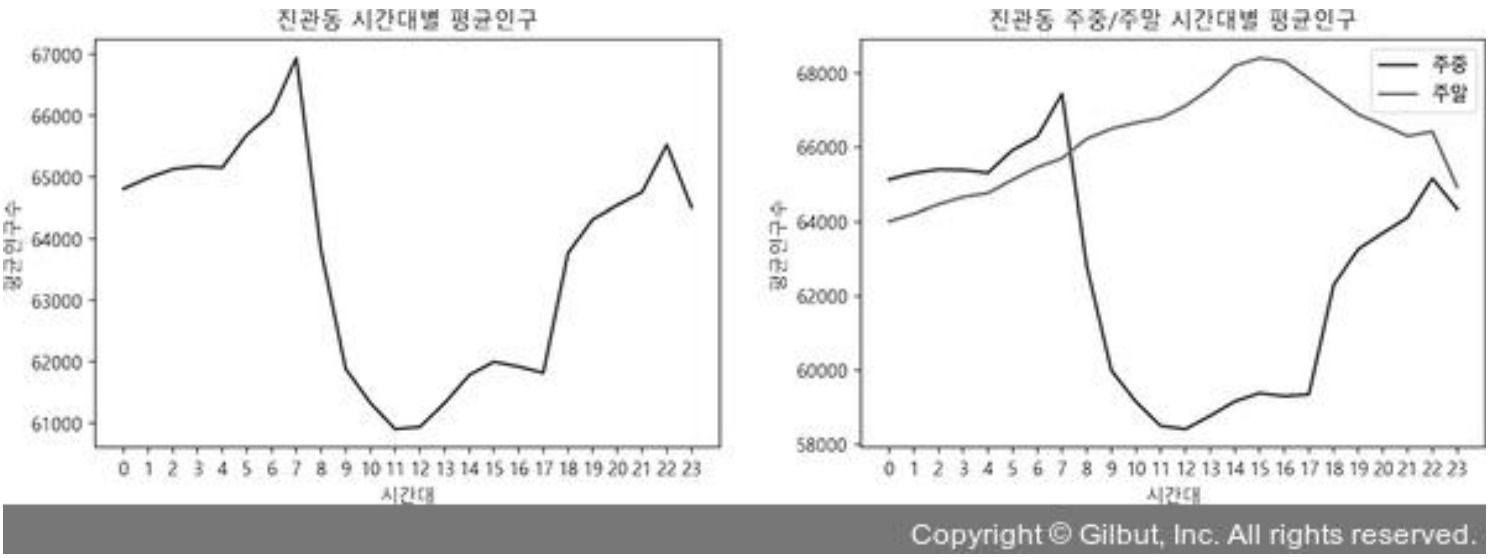


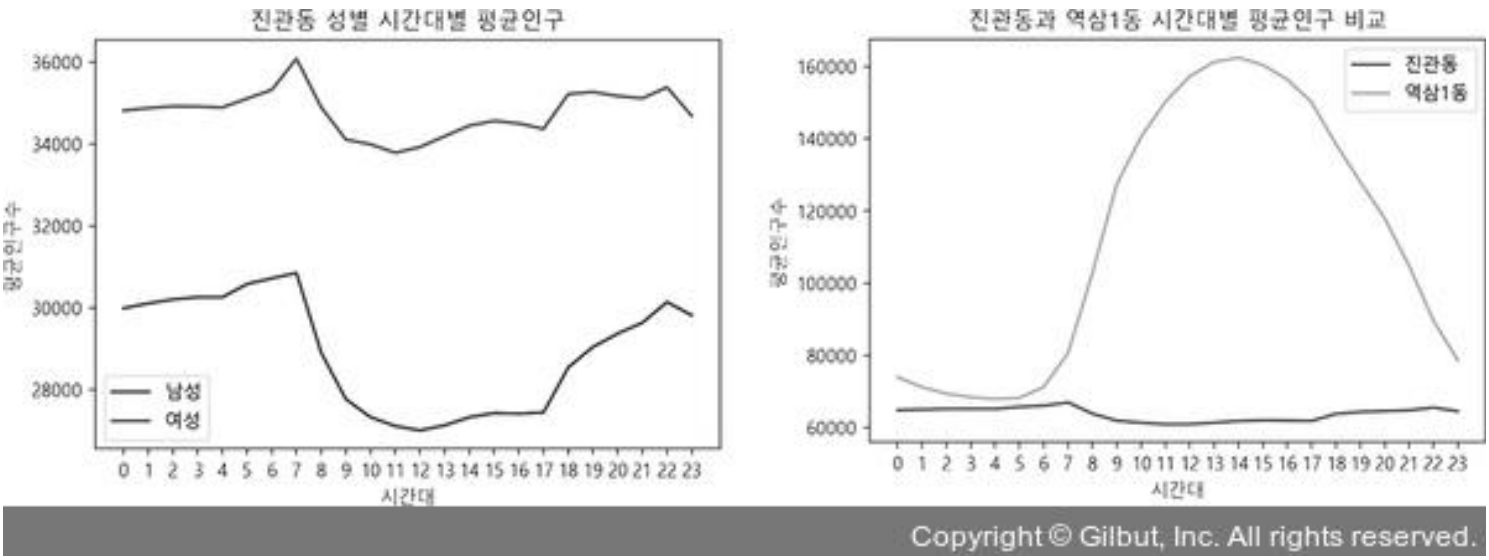
과정이 좀 길긴 했지만, 알고리즘을 작성한 것처럼 단계별로 나누어 진행하면 더 체계적이고 쉽게 프로그래밍할 수 있습니다.

12.2.3 결과 분석하기

결과를 분석해 봅시다. 인구수 상위 10개 지역의 분포를 보면 강남구에 몰려 있네요. 무려 4개의 행정동이 순위에 올랐습니다. 서울시에서 인구가 가장 많은 행정동은 역삼1동이고, 여의도동, 서교동, 종로 1,2,3,4가동 순입니다. 이들은 서울의 3대 중심업무지구인 KBD(Kangnam Business District), YBD(Yeouido Business District), CBD(Central Business District)에 속합니다.

조금 재미있는 것은 은평구 진관동입니다. 다른 지역은 사람이 많은 지역임을 익히 알고 있었지만, 진관동이 순위에 꼽을 정도로 인구가 많을 줄은 몰랐어요. 진관동을 11장에서 작성한 핫플레이스 프로그램에 넣어 분석해 보면 다음과 같은 그래프가 나타납니다.





▲ 그림 12-17 진관동 분석 결과

11장에서 분석해 본 압구정동과는 확연히 다른 모양의 그래프가 나타납니다. 진관동의 평균인구는 출근시간 이후에 급격히 줄고 퇴근시간 이후에 급격히 늘어나네요. 업무지구가 아닌 주거지구의 역할이 크다고 볼 수 있습니다. 주중/주말 그래프를 보면 이 점이 더 뚜렷해집니다. 주말에는 주중과 비교해 낮 시간대에 인구수가 매우 많습니다. 가족과 집이나 동네에서 시간을 많이 보내는 것 같죠?

남녀 인구 그래프도 흥미롭습니다. 남성보다 여성이 압도적으로 많네요. 그런데 출근 시간대에 여성인구가 줄어드는 폭보다 남성인구가 줄어드는 폭이 훨씬 큽니다. 아마 맞벌이 부부는 모두 출근하고, 전업주부인 여성은 출근하지 않기 때문이 아닐까 추측할 수 있습니다.

인구수 1위 역삼1동과 비교해 봅시다. 모든 시간대에서 역삼1동이 진관동보다 인구수가 많군요. 업무지구인 역삼1동은 출근시간부터 인구가 꾸준히 늘어나고 퇴근시간에는 인구가 줄어드는 반면, 진관동은 반대 현상이 나타납니다. 업무지구와 주거지구의 차이를 그래프로 확인할 수 있습니다.

다른 관점에서도 앞의 결과를 분석해 보세요. 그동안 보이지 않던 것이 보일지도 모릅니다.

12.2.4 추가 분석: 남성이 많은 지역과 여성이 많은 지역 비교하기

지금까지 진행한 코드를 이해했다면 남성이 많은 지역 상위 10곳, 여성이 많은 지역 10곳도 각각 지도에 표시할 수 있습니다. 남성이나 여성이 많은 지역 10곳을 지도 위에 마커로 표시하되 1~5위 지역과 6~10위 지역을 다른 색으로 표시해 보겠습니다. 앞서 작성한 알고리즘을 조금만 수정하면 남성 또는 여성 인구만을 기준으로 프로그램을 작성할 수 있어요. 총생활인구를 구하는 대신, 남성 인구와 여성 인구의 합을 구하는 것으로 대체하면 됩니다. 변경된 부분만 붉게 표시해 보겠습니다.

하위 목표 5

인구가 많은 상위 10개 행정동을 뽑아 지도에 표시하기

알고리즘

1. 필요한 데이터 파일을 읽어 들이기
2. 행정동별 남녀 인구수를 저장하는 빈 딕셔너리를 만들고 행정동명을 키로, 인구수를 값으로 저장하기
3. 인구수를 기준으로 내림차순 정렬하고, 앞에서 10개 항목만 행정동코드 리스트에 저장하기
4. 행정동코드 리스트의 8자리 행정동코드에 해당하는 행정동명을 찾아서 행정동명 리스트에 저장하기
5. 행정동명 리스트의 행정동명에 해당하는 위도와 경도를 찾아서 각각 위도, 경도 리스트에 저장하기
6. 완성된 리스트로 지도를 그리고 1~5위 지역과 6~10위 지역을 다른 색으로 표시하기

먼저 남성 인구를 기준으로 작성해 보겠습니다. 알고리즘이 거의 같아서 코드도 바뀔 부분이 많지 않습니다. 리스트명이 `top10_code8`에서 `male_code8`로 변경되고 남성 인구를 구하는 코드를 추가하면 됩니다. 남성 인구를 구하는 코드는 11장에서 배운 바 있죠. 인구 데이터인 `data`의 인덱스 `[4]`부터 `[17]`까지의 합을 구하면 됩니다. 그럼 변경된 부분만 작성해 보겠습니다.

```
# 2. 인구 데이터를 읽으며 행정동별 남성 인구수를 저장하는 딕셔너리 완성하기
dong_population = {}
for row in data:
    # 남성 인구수를 저장하는 변수 p
    dong_code, p = row[2], sum(row[4:18])
…
# 6. 완성된 리스트로 지도 그리기
import folium

for i in range(len(top10_name)):
    top10_name[i] = str(i + 1) + '. ' + top10_name[i]

map_top10 = folium.Map([top10_lat[0], top10_long[0]], zoom_start=12)
for i in range(10):
    # 1~5위 지역은 진한 파란색>, 6~7위 지역은 연한 파란색으로 표시하기
    if i < 5:
        folium.Marker(
            [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
            icon=folium.Icon(color='darkblue', icon='male', prefix='fa')
            ).add_to(map_top10)
    else:
        folium.Marker(
            [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
            icon=folium.Icon(color='lightblue', icon='male', prefix='fa')
            ).add_to(map_top10)
map_top10
```

실 행 결 과



코드에서 변경된 부분은 앞에서 배운 내용이라 설명을 생략하겠습니다. 지도 그리는 부분만 짚고 넘어가 봅시다. 1~5위 지역과 그 외 지역은 색상을 달리하기로 했죠. 그래서 마커를 추가하는 반복문 내부에 `i`가 5 이하인 경우와 그렇지 않은 경우를 분리해 색상을 다르게 했습니다. 그리고 남성임을 나타내는 아이콘을 https://fontawesome.com/v4.7.0/icons에서 찾아 아이콘 모양을 `icon='male'`로 변경하고 `prefix='fa'` 옵션도 추가했습니다.

결과를 보니 앞서 전체 인구를 대상으로 분석한 것과 큰 차이는 없는 것 같군요. 강남에 4곳과 중심업무지구, 진관동까지 앞에서 봤던 지역입니다. 다만 인구수에 따라 색상을 달리하니 남성인구가 더 많은 곳이 보입니다. 확실히 서울 도심과 중심업무지구에 남성인구가 많네요.

이번에는 여성이 많은 상위 10개 지역을 지도에 표시해 봅시다. 앞에서 진행한 것처럼 1~5위 지역은 빨간색으로, 6~10위 지역은 분홍색으로 표시하겠습니다. 앞의 코드에서 리스트명의 `male` 부분을 `female`로만 변경하면 여성 데이터를 저장하는 새로운 리스트를 만들 수 있습니다.

```
# 2. 인구 데이터를 읽으며 행정동별 여성 인구수를 저장하는 딕셔너리 완성하기
dong_population = {}
for row in data:
    # 여성 인구수를 저장하는 변수 p
    dong_code, p = row[2], sum(row[18:32])
…
# 6. 완성된 리스트로 지도 그리기
import folium

for i in range(len(top10_name)):
    top10_name[i] = str(i + 1) + '. ' + top10_name[i]

map_top10 = folium.Map([top10_lat[0], top10_long[0]], zoom_start=12)
for i in range(10):
    # 1~5위 지역은 빨간색, 6~7위 지역은 분홍색으로 표시하기
    if i < 5:
        folium.Marker(
            [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
            icon=folium.Icon(color='red', icon='female', prefix='fa')
            ).add_to(map_top10)
    else:
        folium.Marker(
            [top10_lat[i], top10_long[i]], tooltip=top10_name[i],
            icon=folium.Icon(color='pink', icon='female', prefix='fa')
            ).add_to(map_top10)
map_top10
```

실행결과



Copyright © Gilbut, Inc. All rights reserved.

여성이 많은 지역을 표시하니 재미있는 결과가 나타납니다. 여성이 많은 지역 상위 5곳은 대부분 남성이 많은 지역 상위 5곳과 겹칩니다. 진관동을 제외하고요. 진관동은 기본적으로 인구가 많은 지역이고, 남성보다는 여성이 더 많은 것으로 나타납니다. 그리고 남성 인구 지도에서는 보이지 않던 지역이 등장했습니다. 바로 송파구의 잠실3동(9위)과 강동구 길동(10위)입니다. 그리고 남성 인구 지도에서 6위를 차지한 금천구 가산동은 여성 인구 지도에서는 나타나지 않는군요.

종합해 보면, 여성과 남성의 생활인구가 많은 지역은 비슷합니다. 그러나 남성 생활인구가 여성 생활인구보다 서울의 서쪽 지역(금천구 가산동)에서 조금 더 많고, 여성 생활인구는 서울의 동쪽 지역(송파구 잠실3동, 강동구 길동)에서 더 많습니다. 성별 분포의 차이가 나타나는 이유는 정확히 설명하기는 어렵지만, 각 지역에 많이 분포하는 업종이 어떤 것이냐에 영향이 있지 않을까요?

지금까지 진행한 데이터와 코드를 참고하면 연령별 인구 분석도 가능하겠죠. 시간대별 분석을 그래프가 아닌 지도에 표시할 수도 있을 겁니다. 본인이 원하는 대로 이리저리 프로그램을 작성해 보고 결과를 분석하다 보면 프로그래밍에 성취감을 느낄 수 있을 거예요.

벌써 Part 2까지 배웠네요. Part 1에서는 파이썬 프로그래밍의 기초를 다졌고, Part 2에서는 배운 내용을 응용하는 프로젝트를 진행해 봤습니다. 마지막으로 Part 3에서는 파이썬의 심화 문법을 배워 보겠습니다. 심화 문법을 알아야 이 책이 끝나고도 머신러닝, 딥러닝과 같은 더 복잡하고 어려운 코드를 이해할 수 있습니다. 그리고 우리가 평소에 사용하는 프로그램의 외형을 만드는 방법도 배워 보겠습니다.

12

마무리

1 folium 설치

```
!pip install folium
```

2 folium 지도 만들고 나타내기

형식

```
지도이름 = folium.Map([위도, 경도], zoom_start=지도비율)
지도이름
```

3 folium 마커 만들고 추가하기

형식

```
folium.Marker([위도, 경도]).add_to(지도이름)
```

4 마커에 추가할 수 있는 옵션

툴팁 표현	`tooltip='표현할 텍스트'`
팝업 표현	`popup='표현할 텍스트'`
아이콘	`icon= folium.Icon(color= '아이콘 색상', icon='아이콘 모양')` fontawesome의 아이콘 활용 시 `prefix='fa'` 추가

Self Check

1 `folium`을 활용하여 해운대 해수욕장을 중심으로 하는 지도를 표시하고, fontawesome의 hand-o-down을 사용해 마커를 추가하세요. `zoom_start`는 11로 설정합니다.

위치명	위도	경도	마커 아이콘(fontawesome)
해운대 해수욕장	35.15870	129.16047	hand-o-down

2 다음 위치를 모두 `folium`으로 나타내세요. 마커에 마우스를 올리면 위치명이 나오게 툴팁을 설정하세요.

위치명	위도	경도	마커 아이콘(fontawesome)
해운대 해수욕장	35.15870	129.16047	hand-o-down
부산대학교	35.23360	129.08309	university
김해국제공항	35.17357	128.94632	plane

정답 및 해설